

A Multimodal RAG System for Personalized Food Recommendations: Preliminary Report

Divyansh Agarwal
Ms in Applied Data Science
University of Florida
divyansh.agarwal@ufl.edu

Abstract—The restaurant aggregator industry increasingly relies on technology to enhance user experience. Traditional recommendation systems, often limited to textual data, fail to capture nuanced user preferences related to the visual appeal of food. This project implements a multimodal Retrieval-Augmented Generation (RAG) system that processes both textual descriptions and visual content to provide personalized food recommendations. This report details the system’s architecture, initial implementation using Amazon Bedrock and FAISS, the Streamlit-based interface prototype, and early evaluation results. We also discuss key challenges, such as semantic query ambiguity, and outline next steps for refinement, including query enhancement and document reranking.

I. PROJECT SUMMARY

This project aims to design and implement a Multimodal Retrieval-Augmented Generation (RAG) system for a restaurant aggregator application. The primary purpose is to enhance food recommendations by integrating and processing both textual data (descriptions, ingredients, nutritional info) and visual content (dish images). By leveraging multimodal data, the system can better capture user preferences and dietary needs, moving beyond the limitations of traditional text-only recommendation engines.

Since the initial project proposal (Deliverable 1), we have implemented the complete data processing pipeline and a functional prototype. This includes:

- **Data Ingestion:** Fetching menu data and images from an S3 bucket.
- **Multimodal Preprocessing:** Implementing a pipeline to generate rich, descriptive summaries for all dish images using a multimodal LLM (Anthropic Claude 3 Sonnet on Amazon Bedrock).
- **Vector Database Creation:** Combining these image descriptions with structured metadata (ingredients, calories, etc.) into a comprehensive `full_description` for each item, which is then embedded using Amazon Titan Embeddings and stored in a FAISS vector index.
- **Interface Prototype:** Building an interactive chat interface using Streamlit (`app.py`) that allows users to query the system with text and image uploads.
- **Query Enhancement:** Implementing an early version of query refinement (`enhance_search`) to improve semantic search relevance.

What remains to be improved is the robustness of the retrieval system. Early evaluations show that simple vector

search can fail on ambiguous queries. Future work, as detailed in Section VI, will focus on advanced document reranking, implementing conversational memory, and further polishing the user interface.

II. SYSTEM ARCHITECTURE AND PIPELINE

The system is designed as a pipeline that transforms raw restaurant data into an interactive, queryable recommendation service. The data flows through four primary stages, as illustrated in Fig. 1 and described below.

Architecture

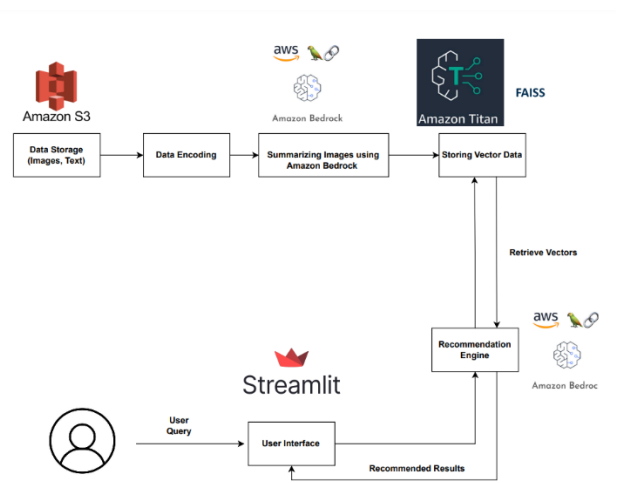


Figure 1. High-level system architecture showing the flow of data from storage to the end-user interface.

- 1) **Data Storage (S3):** The pipeline begins with raw data stored in an Amazon S3 bucket. This includes a CSV file (`restaurants_menu_data.csv`) containing menu item metadata and a corresponding directory of dish images.
- 2) **Data Preprocessing (Bedrock):** This is the core data-building phase.
 - Images are fetched from S3 and encoded into Base64.
 - The `describe_image` function sends the image to the Claude 3 Sonnet model via the Bedrock API

to generate a detailed, savory description of the food.

- This generated description is combined with structured metadata (ingredients, nutrition, price, etc.) to create a single `full_description` text block for each menu item.

3) Model (Embeddings + RAG):

- **Indexing:** The `full_description` of each item is passed to the Amazon Titan Embeddings model. The resulting high-dimensional vectors are indexed and stored in a FAISS (Facebook AI Similarity Search) vector store, which is saved locally as `faiss_index`.
- **Retrieval:** When a user query is received, it is first embedded. FAISS performs an efficient similarity search to retrieve the top-k most relevant documents (menu items).
- **Generation:** These retrieved documents are passed as context to the generative LLM (Claude 3 Sonnet), which synthesizes a final, conversational response or recommendation.

- 4) **Interface (Streamlit):** The user interacts with the system via a Streamlit application (`app.py`). The interface handles text input, image uploads, and renders the chat history, including the final formatted recommendations and images.

III. MODEL IMPLEMENTATION DETAILS

The implementation relies heavily on the AWS ecosystem and the LangChain library to orchestrate the models and data flow.

A. Frameworks and Libraries

The core of the project is built with Python 3.10 and the following key libraries:

- **AWS Interaction:** `boto3` is used for all communication with AWS services, specifically S3 and Bedrock.
- **Orchestration:** `langchain` (specifically `langchain_community`) provides the abstractions for interfacing with the Bedrock LLMs (`BedrockChat`) and embeddings (`BedrockEmbeddings`), as well as for managing documents.
- **Vector Storage:** `faiss-cpu` is used to create, save, and load the local vector index.
- **Data Handling:** `pandas` is used for initial data loading and manipulation from the CSV.
- **Web Interface:** `streamlit` and `streamlit_chat` are used to build the interactive prototype.

B. Model and Training Setup

This system does not involve training a model from scratch. Instead, it "trains" a RAG system by building a comprehensive knowledge base (the FAISS index).

- **Generative/Vision Model:** We use `anthropic.claude-3-sonnet-20240229-v1:0` via Bedrock. This multimodal model is used for two

key tasks: (1) generating descriptions for menu images and (2) synthesizing the final RAG response. We use deterministic settings (`temperature: 0.0`) to ensure reproducible descriptions.

- **Embedding Model:** We use `amazon.titan-embed-text-v2:0` via Bedrock. This model converts our `full_description` text into 1024-dimension vectors for storage in FAISS.

C. Key Functions for Reproducibility

The system's logic is encapsulated in several key functions (primarily in `utils.py`) that ensure a reproducible pipeline:

- `describe_image()`: Takes a Base64-encoded image and uses a system prompt to instruct Claude 3 Sonnet to generate a description focused only on the food, its appearance, and ingredients.
- `enhance_search()`: A query-expansion function inspired by HyDE (Hypothetical Document Embeddings). It uses an LLM to pre-process the user's raw query (e.g., "sweet dish") into a richer set of keywords ("desserts, pastries, cakes, sweet, sugar") to improve retrieval accuracy (see Sec. V).
- `relevance_checker()`: A critical evaluation step. After retrieval, this function passes each retrieved document and the user's query to the LLM and asks a simple "Yes/No" question: "Is this dish relevant...?" This filters out semantically close but contextually irrelevant results (e.g., "Sweet and Sour Pork" for a "sweet dish" query).
- `assistant()`: The final RAG prompt, which instructs the LLM to act as a helpful assistant, determine if the user wants a recommendation or is asking a general question, and format its output as a JSON object with `"recommendation": "yes/no"` and `"response": "..."`.

IV. INTERFACE PROTOTYPE

The interface is a fully functional web application built with Streamlit (see `app.py`). It serves as a prototype for user interaction and validation of the RAG pipeline.

A. Design and Functionality

The interface (Fig. 2) is a simple, single-page chat application.

- **Chat History:** The main area displays the conversation history, with user messages and assistant responses styled using `streamlit_chat`.
- **Text Input:** A persistent text input box at the bottom allows the user to type their query.
- **Image Upload:** A file uploader widget allows the user to optionally add an image to their query. If an image is uploaded, it is encoded and used to generate a description that is appended to the text query, enabling multimodal search (e.g., "recommend something similar to this").
- **Send Button:** A "Send" button triggers the entire RAG pipeline: query enhancement, FAISS search, relevance checking, and response generation.

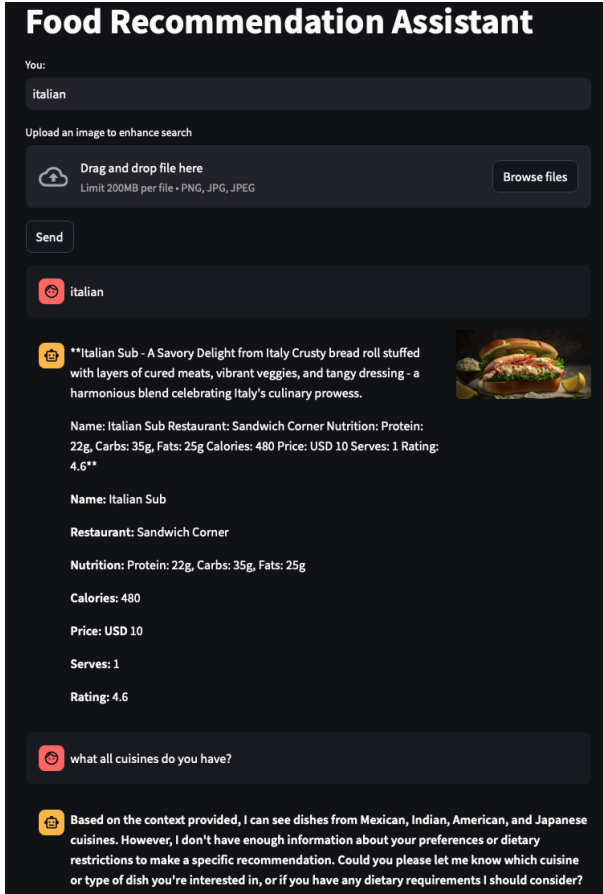


Figure 2. Streamlit chat interface prototype showing the layout of chat messages, text input box, and image uploader widget.

B. Input and Output Format

- **User Input:** A text string (e.g., "I'm looking for a high-protein, low-calorie option") and/or an optional image file (JPG, PNG).
- **System Output:** If the assistant function returns "recommendation": "no", a simple text-based chat response is shown. If it returns "recommendation": "yes", the interface displays a formatted list of up to 3 relevant dishes. For each dish, it displays:
 - The savory summary generated by `dish_summary()`.
 - Key metadata: Restaurant Name, Nutrition, Calories, Price, Serves, and Rating.
 - The corresponding image of the dish, loaded from the local `data/` directory.

C. Current Limitations

The prototype is fully functional but lacks long-term memory. The chat history is maintained via Streamlit's session state, but it is not fed back into the RAG context. This means the model cannot answer follow-up questions (e.g., "Which of those is the cheapest?") or remember past recommendations.

V. EARLY EVALUATION AND RESULTS

Initial evaluation focused on the quality of the vector search retrieval, which forms the foundation of the RAG system. We tested the raw FAISS index with queries of varying ambiguity.

A. Initial Retrieval Performance

We first tested the `db.similarity_search_with_score()` function. The results, summarized in Table I, were mixed.

- **Success Case:** A query for "italian dishes" successfully retrieved relevant items like "Italian Sub" (Score: 1.21) and "Margherita Pizza" (Score: 1.28). This confirmed that the embeddings correctly grouped items by cuisine.
- **Failure Case:** A semantically ambiguous query for "sweet dishes" failed. The top result was "Sweet and Sour Pork" (Score: 1.41). While the word "sweet" is present, this is not a dessert and fails to capture user intent.

This failure demonstrated that naive vector search is insufficient and led to the implementation of a query enhancement and filtering layer.

Table I
EARLY EVALUATION OF NAIVE VECTOR SEARCH

Query String	Top Result Retrieved	Interpretation
"italian dishes"	"Italian Sub" (Score: 1.21)	Success
"sweet dishes"	"Sweet and Sour Pork" (Score: 1.41)	Failure

B. Results after Refinement

We introduced two key refinements:

- 1) **Query Enhancement:** The `enhance_search` function expands a user's query. For example, "dishes with high protein and low calories" is expanded into a keyword-rich query: "high protein low calorie dishes lean proteins... tofu dishes lentil dishes..."
- 2) **Relevance Filtering:** The `relevance_checker` function filters the retrieved results using the LLM.

With these refinements, the system's performance on ambiguous queries improved significantly. A search for "dishes with high protein and low calories" (using the enhanced query) now correctly retrieves and filters results to show:

- Tempura (10g Protein, 180 Cals)
- Miso Soup (5g Protein, 70 Cals)
- Veggie Sandwich (10g Protein, 250 Cals)

These results, while not all "high" protein, are correctly identified as the lowest-calorie and highest-protein options *available in the limited dataset* (50 items), demonstrating a much more intelligent retrieval process.

VI. CHALLENGES AND NEXT STEPS

The primary challenge encountered was the semantic ambiguity of user queries, as demonstrated by the "sweet dishes" failure. Our current query enhancement and relevance-checking functions are effective but add latency, as they require extra LLM calls.

Based on the "Try it out" section of the project notebook, our planned next steps before Deliverable 3 are:

- 1) **Full S3 Integration:** Refactor `app.py` to fetch data and images directly from the S3 bucket at runtime, rather than relying on a local `data/` folder. This is crucial for deployment.
- 2) **Implement Document Reranking:** The current system retrieves by vector relevance and filters. It does not *rerank*. We will implement a reranking step that can re-order the filtered list based on explicit user preferences mentioned in the query (e.g., "high average rating," "low price," or "low calorie").
- 3) **Incorporate Conversational Memory:** Enhance the chatbot by adding `ConversationBufferWindowMemory`. This will allow the LLM to "remember" the last few exchanges, enabling it to answer follow-up questions and avoid recommending the same dishes repeatedly.
- 4) **EC2 Deployment:** Package the application and deploy it on an Amazon EC2 instance to create a publicly accessible prototype.

VII. RESPONSIBLE AI REFLECTION

During implementation, several potential Responsible AI (RAI) issues emerged, which will be a focus of our refinement phase.

- **Fairness and Bias:** The recommendation model could inadvertently introduce bias. For instance, if the LLM's image descriptions are more appealing for Western dishes than for non-Western dishes, the RAG system might develop a bias. Furthermore, the model could create a "rich-get-richer" loop, over-recommending items that already have high ratings. Our reranking (Next Step 2) must be designed to mitigate this, perhaps by introducing stochasticity or promoting less-reviewed items.
- **Privacy:** The system is designed to handle sensitive user data, specifically dietary restrictions and allergies (e.g., "I'm allergic to peanuts"). As we implement conversational memory (Next Step 3), we must ensure this sensitive health-related information is not stored insecurely or used for any purpose other than the user's immediate request.
- **Transparency:** The system's recommendations are the result of a complex, multi-stage process (embedding, retrieval, LLM filtering, LLM generation). This can be opaque to the user. We plan to improve transparency by having the `dish_summary` function explicitly state *why* a dish was recommended (e.g., "You asked for high protein, and this has 20g...").

Addressing these challenges will be critical as we move from a functional prototype to a robust, user-centric application.